

```
import itertools
import numpy as np
```

```
def bipolar_activation(net, lambdada = 1):
    return (2/(1+np.exp(-lambdada*net))) - 1
```

```
def unipolar_activation(net, lambdada = 0.3):
    return (1/(1+np.exp(-lambdada*net)))
```

```
def bipolar_fdash(y_pred):
    return (1/2)*(1-(y_pred**2))
```

```
def unipolar_fdash(y_pred):
    return (y_pred*(1-y_pred))
```

```
def predict(X, weights, type):
    net = np.dot(X, weights)
    if type == 'unipolar':
        return unipolar_activation(net)
    elif type == 'bipolar':
        return bipolar_activation(net)
```

```
def calculate_r(yi, y_pred, type):
    r = yi - y_pred
    if type == 'unipolar':
        return r*unipolar_fdash(y_pred)
    elif type == 'bipolar':
        return r*bipolar_fdash(y_pred)
```

```
def train(X, y, weights, type, epochs=500, c=0.5):
    for epoch in range(epochs):
        loss = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights, type)
            r = yi - y_pred
            loss += abs(r)
            delta_w = c*r*Xi
            weights += delta_w

        if (epoch+1)%50 == 0 :
            print(f'Weights after epoch {epoch+1} : ',weights)
        if loss == 0:
            break

    print('Learned weights : ', weights)
    weights = weights.reshape((n + 1, 1))
    test(X, y, weights, type)
```

```
def test(X, y, learned_weights, type):
    nets = np.dot(X,learned_weights).flatten()
    print('Actual Values : ',y)
    if type == 'unipolar':
        y_pred = np.array([unipolar_activation(net) for net in nets])
        print('Predicted Values : ', y_pred)
    else:
        y_pred = np.array([bipolar_activation(net) for net in nets])
        print('Predicted Values : ', y_pred)
```

```
n = int(input('Enter number of bits : '))
X = np.array([list(i) + [1] for i in itertools.product([0,1], repeat = n)])

weights = input(f'Enter initial {n} weights and 1 bias : ')
weights = np.array([float(weight) for weight in weights.split()], dtype='longdouble')
print()
```

```
↗ Enter number of bits : 2
Enter initial 2 weights and 1 bias : 0 0.5 1.5
```

```
# 1) AND GATE UNIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
print()
print()
```



```
---- AND GATE USING PERCEPTRON ----
Weights after epoch 50 : [ 4.89960089  4.86039897 -8.06675761]
Weights after epoch 100 : [ 7.81418244  7.71841895 -12.23733586]
Weights after epoch 150 : [ 9.81130365  9.71988871 -15.19797992]
Weights after epoch 200 : [ 11.35691891  11.27705668 -17.51328962]
Weights after epoch 250 : [ 12.62369771  12.55441883 -19.41786867]
Weights after epoch 300 : [ 13.69815348  13.63752318 -21.03553605]
Weights after epoch 350 : [ 14.63106816  14.57739373 -22.44080774]
Weights after epoch 400 : [ 15.45519594  15.40715619 -23.68236273]
Weights after epoch 450 : [ 16.1930277  16.14961222 -24.79385809]
Weights after epoch 500 : [ 16.86071934  16.82115183 -25.79956539]
Learned weights : [ 16.86071934  16.82115183 -25.79956539]
Actual Values : [0 0 0 1]
Predicted Values : [4.34939051e-04 6.33565692e-02 6.40646416e-02 9.14094942e-01]
Predicted Values : [4.34939051e-04 6.33565692e-02 6.40646416e-02 9.14094942e-01]
```

```
# 2) OR GATE UNIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = 0
train(X, y, weights.copy(), 'unipolar')
print()
print()
```



```
---- OR GATE USING PERCEPTRON ----
Weights after epoch 50 : [ 6.84739736  7.06570149 -0.92512945]
Weights after epoch 100 : [10.28547842 10.41514962 -3.11264477]
Weights after epoch 150 : [12.67841579 12.76861613 -4.48250051]
Weights after epoch 200 : [14.49589122 14.56450972 -5.47796602]
Weights after epoch 250 : [15.95385272 16.00903472 -6.2584005 ]
Weights after epoch 300 : [17.16760461 17.21366907 -6.89914697]
Weights after epoch 350 : [18.20538396 18.24487595 -7.44193977]
Weights after epoch 400 : [19.11069388 19.14523229 -7.91232581]
Weights after epoch 450 : [19.91287196 19.94354758 -8.32706953]
Weights after epoch 500 : [20.63259052 20.66017199 -8.69775555]
Learned weights : [20.63259052 20.66017199 -8.69775555]
Actual Values : [0 1 1 1]
Predicted Values : [0.06854058 0.97310954 0.97289217 0.99994335]
Predicted Values : [0.06854058 0.97310954 0.97289217 0.99994335]
```

```
# 3) NOR GATE UNIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'unipolar')
print()
print()
```



```
---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50 : [-7.35187773 -7.27869603 1.18705941]
Weights after epoch 100 : [-10.59039761 -10.59143668 3.25648215]
Weights after epoch 150 : [-12.89074593 -12.91180078 4.58209642]
Weights after epoch 200 : [-14.6570042 -14.68331143 5.55397003]
Weights after epoch 250 : [-16.08296541 -16.10987863 6.31971719]
Weights after epoch 300 : [-17.27500067 -17.30096027 6.95045221]
Weights after epoch 350 : [-18.29714831 -18.32166288 7.48599424]
Weights after epoch 400 : [-19.19070242 -19.2136785 7.95089387]
Weights after epoch 450 : [-19.98373587 -20.00523071 8.36134595]
Weights after epoch 500 : [-20.69614797 -20.71627182 8.72858602]
Learned weights : [-20.69614797 -20.71627182 8.72858602]
Actual Values : [1 0 0 0]
Predicted Values : [9.32047561e-01 2.66928039e-02 2.68500999e-02 5.51637073e-05]
Predicted Values : [9.32047561e-01 2.66928039e-02 2.68500999e-02 5.51637073e-05]
```

```
# 4) NAND GATE UNIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = 0
train(X, y, weights.copy(), 'unipolar')
print()
print()
```



```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50 : [-5.2757023 -5.05882024 8.47369195]
Weights after epoch 100 : [-8.01935903 -7.88040207 12.50774125]
Weights after epoch 150 : [-9.95547165 -9.8510645 15.40302437]
```

```

Weights after epoch 200 : [-11.4698824 -11.38555585 17.67892493]
Weights after epoch 250 : [-12.71722857 -12.6463288 19.55684032]
Weights after epoch 300 : [-13.77818071 -13.71699348 21.15518701]
Weights after epoch 350 : [-14.7010705 -14.64726743 22.54579636]
Weights after epoch 400 : [-15.51742374 -15.46943328 23.77584195]
Weights after epoch 450 : [-16.24903531 -16.20574127 24.87806481]
Weights after epoch 500 : [-16.91163156 -16.87221081 25.87614494]
Learned weights : [-16.91163156 -16.87221081 25.87614494]
Actual Values : [1 1 1 0]
Predicted Values : [0.99957494 0.93709625 0.93639552 0.08530878]
Predicted Values : [0.99957494 0.93709625 0.93639552 0.08530878]

```

```

# 5) AND GATE BIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'bipolar')
print()
print()

```

```

---- AND GATE USING PERCEPTRON ----
Weights after epoch 50 : [ 4.73577324  4.62750522 -7.01349945]
Weights after epoch 100 : [ 5.91894117  5.86134955 -8.90793249]
Weights after epoch 150 : [ 6.65257211  6.61328127 -10.05209717]
Weights after epoch 200 : [ 7.18663818  7.15680113 -10.87590836]
Weights after epoch 250 : [ 7.60705921  7.58300092 -11.52046772]
Weights after epoch 300 : [ 7.9539111  7.93375268 -12.05016437]
Weights after epoch 350 : [ 8.24919186  8.23184344 -12.49988188]
Weights after epoch 400 : [ 8.50629441  8.49106736 -12.89067197]
Weights after epoch 450 : [ 8.7339843  8.72041569 -13.23622549]
Weights after epoch 500 : [ 8.93831425  8.92607788 -13.54595075]
Learned weights : [ 8.93831425  8.92607788 -13.54595075]
Actual Values : [-1 -1 -1 1]
Predicted Values : [-0.99999738 -0.98048421 -0.98024632  0.97370896]

```

```

# 6) OR GATE BIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = -1
train(X, y, weights.copy(), 'bipolar')
print()
print()

```

```

---- OR GATE USING PERCEPTRON ----
Weights after epoch 50 : [ 5.4501524  5.52174958 -2.20262292]
Weights after epoch 100 : [ 6.85143995  6.8893436 -2.93666598]
Weights after epoch 150 : [ 7.6732892  7.69903063 -3.35824581]
Weights after epoch 200 : [ 8.25587622  8.27535868 -3.65475106]
Weights after epoch 250 : [ 8.70725662  8.72292683 -3.88352638]
Weights after epoch 300 : [ 9.07569117  9.08879626 -4.06978147]
Weights after epoch 350 : [ 9.38693306  9.3981944 -4.22684818]
Weights after epoch 400 : [ 9.6563492  9.66622146 -4.36263495]
Weights after epoch 450 : [ 9.89384567  9.90263382 -4.48221871]
Weights after epoch 500 : [10.10618177 10.11410031 -4.58905294]
Learned weights : [10.10618177 10.11410031 -4.58905294]
Actual Values : [-1 1 1 1]
Predicted Values : [-0.97987952  0.99206029  0.99199742  0.99999967]

```

```

# 7) NOR GATE BIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'bipolar')
print()
print()

```

```

---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50 : [-5.51486091 -5.57631294  2.23504423]
Weights after epoch 100 : [-6.88324901 -6.91847508  2.95253726]
Weights after epoch 150 : [-7.69430107 -7.7188361  3.36872883]
Weights after epoch 200 : [-8.27154731 -8.29034745  3.66257136]
Weights after epoch 250 : [-8.71974636 -8.73497871  3.88976057]
Weights after epoch 300 : [-9.08607098 -9.09887156  4.07496354]
Weights after epoch 350 : [-9.39581166 -9.40684909  4.23128149]
Weights after epoch 400 : [-9.66410533 -9.67380606  4.3665083 ]
Weights after epoch 450 : [-9.90073092 -9.9093835  4.48565753]
Weights after epoch 500 : [-10.11237173 -10.12018044  4.59214478]
Learned weights : [-10.11237173 -10.12018044  4.59214478]
Actual Values : [ 1 -1 -1 -1]

```

Predicted Values : [0.97994101 -0.99208388 -0.99202207 -0.99999968]

```
# 8) NAND GATE BIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = -1
train(X, y, weights.copy(), 'bipolar')
print()
print()
```

```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50 : [-4.77218557 -4.6659048  7.07298404]
Weights after epoch 100 : [-5.93888316 -5.8818855  8.93926929]
Weights after epoch 150 : [-6.666349  -6.62733635 10.07343109]
Weights after epoch 200 : [-7.19717092 -7.16749452 10.8920946 ]
Weights after epoch 250 : [-7.61558762 -7.59163381 11.53351262]
Weights after epoch 300 : [-7.96107743 -7.94099238 12.06109128]
Weights after epoch 350 : [-8.25537198 -8.2380779  12.50928361]
Weights after epoch 400 : [-8.51172733 -8.49654214 12.89892277]
Weights after epoch 450 : [-8.73883145 -8.72529607 13.24357678]
Weights after epoch 500 : [-8.9426898  -8.93048046 13.55257963]
Learned weights : [-8.9426898  -8.93048046 13.55257963]
Actual Values : [ 1  1  1 -1]
Predicted Values : [ 0.9999974  0.98052719  0.98029034 -0.97376466]
```